Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 5
з дисципліни: «Теорія алгоритмів»
Тема: «Алгоритми на графах. Мінімальне кістякове дерево»

Варіант № 15

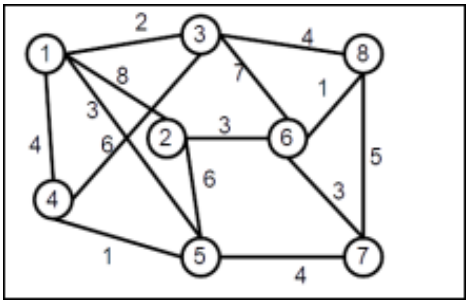
Виконав:
Студент групи АІ-254
Панькевич В. В.
Перевірили:
Смик С. Ю.
Арсірій О.О.

Одеса 2026

Мета роботи: набуття практичних навичок із проєктування, реалізації, тестування та аналізу алгоритмів Пріма (Prim) і Крускала (Kruskal) для побудови мінімального кістякового дерева.

Завдання:

1. Побудувати матриці та списки суміжності для зважених неорієнтованих графів за варіантами завдань.
2. Побудувати за допомогою алгоритму Прима мінімальне кістякове дерево (МКД) для графа за варіантами завдань та показати графічно хід побудови МКД за алгоритмом Пріма (Таблиця 5.1).
3. Запрограмувати алгоритм Пріма. Запустити програму, показати результат за варіантами завдань. Показати, що результати ручної та автоматизованої побудови МКД співпадають.
4. Побудувати за допомогою алгоритму Крускала МКД для графа за варіантами завдань та показати графічно хід побудови МКД за алгоритмом Крускала (Таблиця 5.2).
5. Запрограмувати алгоритм Крускала. Запустити програму, показати результат за варіантами завдань. Показати, що результати ручної та автоматизованої побудови МКД співпадають.
6. Порівняти результати побудови МКД за двома алгоритмами, поясніть різницю, якщо вона буде

Варіант	Граф
15	

Результати виконання завдання:

1. Представлення зваженого графа

Для виконання лабораторної роботи використовується зважений неорієнтований зв'язний граф відповідно до варіанта №15. Граф складається з 8 вершин і 14 ребер, кожному з яких відповідає певне значення ваги.

Математично граф можна представити у вигляді G = (V, E), де V – множина вершин, E – множина ребер графа.

Множина вершин заданого графа: V = {1, 2, 3, 4, 5, 6, 7, 8}.

Графічне представлення зваженого неорієнтованого графа для варіанта №15 наведено на рисунку 5.1.

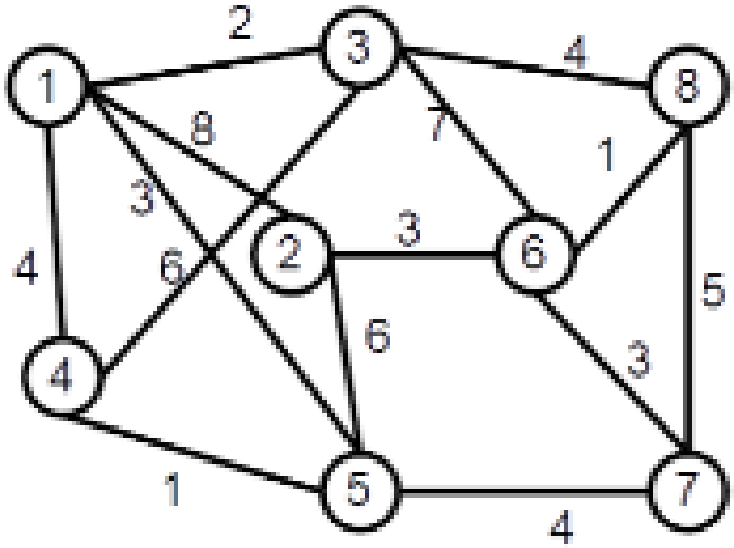


Рисунок 5.1 – Зважений неорієнтований граф варіанта №15

Для представлення графа у вигляді матриці суміжності було використано квадратну матрицю розміром 8 × 8. Кожен її елемент відповідає вазі ребра між двома вершинами. Якщо вершини не з'єднані ребром, у відповідній комірці записується 0.

Оскільки граф є неорієнтованим, матриця симетрична відносно головної діагоналі. Матрицю суміжності з вагами ребер наведено в таблиці 5.1.

Таблиця 5.1 – Матриця суміжності зваженого графа варіанта №15

Вершина	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
1	0	8	2	4	3	0	0	0
2	8	0	0	0	6	3	0	0
3	2	0	0	6	0	7	0	4
4	4	0	6	0	1	0	0	0
5	3	6	0	1	0	0	4	0
6	0	3	7	0	0	0	3	1
7	0	0	0	0	4	3	0	5
8	0	0	4	0	0	1	5	0

Альтернативним способом представлення графа є використання списків суміжності. Для кожної вершини формується список усіх суміжних вершин із зазначенням ваг відповідних ребер. Такий спосіб дозволяє зберігати інформацію лише про наявні зв'язки між вершинами.

Списки суміжності зваженого графа варіанта №15 наведено в таблиці 5.2.

Таблиця 5.2 – Списки суміжності зваженого графа

Вершина	***Суміжні вершини (вага ребра)***
1	2(8), 3(2), 4(4), 5(3)
2	1(8), 5(6), 6(3)
3	1(2), 4(6), 6(7), 8(4)
4	1(4), 3(6), 5(1)
5	1(3), 2(6), 4(1), 7(4)
6	2(3), 3(7), 7(3), 8(1)
7	5(4), 6(3), 8(5)
8	3(4), 6(1), 7(5)

За результатами побудови матриці суміжності (таблиця 5.1) та списків суміжності (таблиця 5.2) отримано два способи представлення заданого зваженого

графа. Обидва представлення містять однакову інформацію про зв'язки між вершинами та ваги ребер.

2. Побудова мінімального кістякового дерева за алгоритмом Прима

Алгоритм Прима (Prim) використовується для побудови мінімального кістякового дерева зваженого зв'язного графа. Він належить до жадібних алгоритмів і починає роботу з довільної вершини, поступово додаючи до дерева ребра з найменшою допустимою вагою.

На кожному кроці вибирається ребро, яке з'єднує одну з уже включених до дерева вершин із вершиною, що ще не входить до нього. Процес продовжується, доки всі вершини графа не будуть включені до мінімального кістякового дерева.

Для реалізації алгоритму Прима використано граф варіанта №15, представлений на рисунку 5.1. Початковою вершиною обрано вершину 1. Псевдокод алгоритму наведено в таблиці 5.3.

Таблиця 5.3 – Псевдокод алгоритму Прима

```
Prim(G)
// Вхідні дані: зважений зв'язний граф G = (V, E)
// Вихідні дані: ET – множина ребер мінімального
// кістякового дерева графа G

v0 ← 1 // Початкова вершина
VT ← {v0} // Вершини, включені до МКД
ET ← ∅ // Ребра мінімального дерева

// МКД повинно містити |V| - 1 ребро
for i ← 1 to |V| - 1 do

    // Знаходимо допустиме ребро мінімальної ваги
    e* ← min_weight_edge(v, u), де
         v ∈ VT
         u ∈ V - VT

    // Включаємо ребро до дерева
    ET ← ET ∪ {e*}

    // Додаємо нову вершину
    VT ← VT ∪ {u}

return ET
```

Позначення:

V_T – вершини, які вже включені до дерева.

E – ребра-кандидати, що з'єднують включені вершини з невключеними.

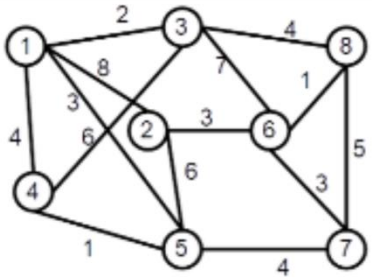
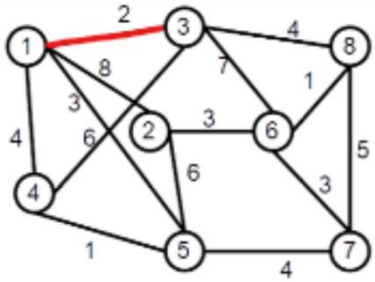
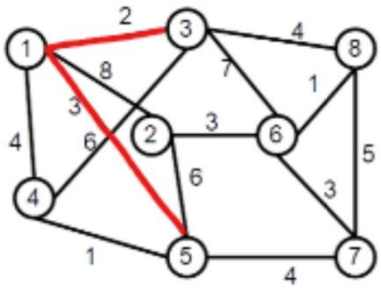
$V-V_T$ – вершини, які ще залишилися поза деревом.

e* – ребро, яке додали на цьому кроці.

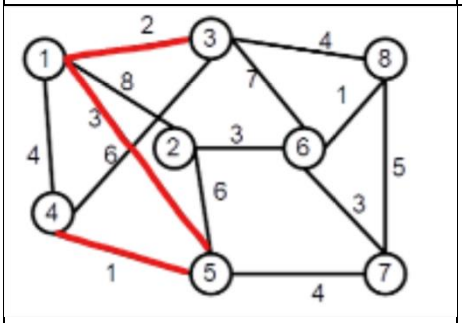
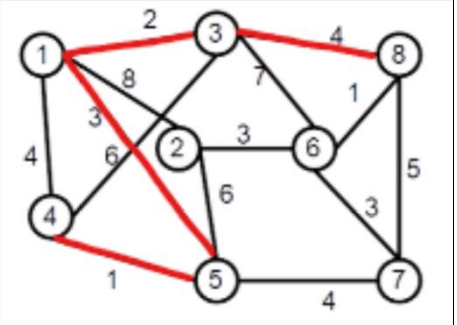
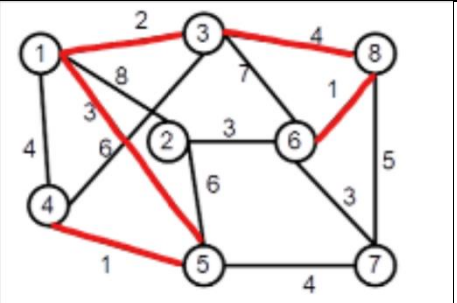
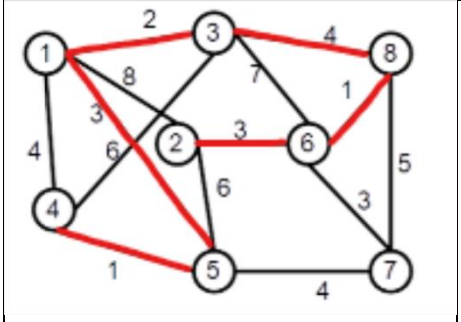
E_T – усі ребра, уже додані до мінімального кістякового дерева.

Таблиця 5.4 – Хід побудови мінімального кістякового дерева за алгоритмом Прима

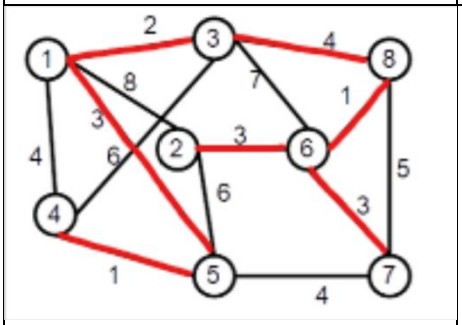
Початок таблиці 5.4

Зображення	V_T	E	$V - V_T$	e*	E_T
	{1}	(1,2) = 8 (1,3) = 2 (1,4) = 4 (1,5) = 3	{2, 3, 4, 5, 6, 7, 8}	–	∅
	{1, 3}	(1,2) = 8 (1,4) = 4 (1,5) = 3 (3,4) = 6 (3,6) = 7 (3,8) = 4	{2, 4, 5, 6, 7, 8}	(1,3) = 2	(1,3) = 2
	{1, 3, 5}	(1,2) = 8 (1,4) = 4 (2,5) = 6 (3,4) = 6 (3,6) = 7 (3,8) = 4 (4,5) = 1 (5,7) = 4	{2, 4, 6, 7, 8}	(1,5) = 3	(1,3) = 2 (1,5) = 3

Продовження таблиці 5.4

Зображення	V_T	E	V - V_T	e*	E_T
	{1, 3, 4, 5}	(1,2) = 8 (2,5) = 6 (3,6) = 7 (3,8) = 4 (5,7) = 4	{2, 6, 7, 8}	(4,5) = 1	(1,3) = 2 (1,5) = 3 (4,5) = 1
	{1, 3, 4, 5, 8}	(1,2) = 8 (2,5) = 6 (3,6) = 7 (5,7) = 4 (6,8) = 1 (7,8) = 5	{2, 6, 7}	(3,8) = 4	(1,3) = 2 (1,5) = 3 (4,5) = 1 (3,8) = 4
	{1, 3, 4, 5, 6, 8}	(1,2) = 8 (2,5) = 6 (2,6) = 3 (5,7) = 4 (6,7) = 3 (7,8) = 5	{2, 7}	(6,8) = 1	(1,3) = 2 (1,5) = 3 (4,5) = 1 (3,8) = 4 (6,8) = 1
	{1, 2, 3, 4, 5, 6, 8}	(5,7) = 4 (6,7) = 3 (7,8) = 5	{7}	(2,6) = 3	(1,3) = 2 (1,5) = 3 (4,5) = 1 (3,8) = 4 (6,8) = 1 (2,6) = 3

Кінець таблиці 5.4

Зображення	V_T	E	V - V_T	e*	E_T
	{1, 2, 3, 4, 5, 6, 7, 8}	∅	∅	(6,7) = 3	(1,3) = 2 (1,5) = 3 (4,5) = 1 (3,8) = 4 (6,8) = 1 (2,6) = 3 (6,7) = 3

Мінімальне кістякове дерево: {(1,3) = 2, (1,5) = 3, (4,5) =1, (3,8) = 4, (6,8) = 1, (2,6) = 3, (6,7) = 3}.

Загальна вага МКД: 2 + 3 + 1 + 4 + 1 + 3 + 3 = 17.

На 4-му кроці ребра (3,8) і (5,7) мають однакову мінімальну вагу 4; обрано (3,8). На 6-му кроці ребра (2,6) і (6,7) мають однакову мінімальну вагу 3; обрано (2,6).

3. Реалізація алгоритму Прима мовою Python

Для перевірки результатів ручної побудови мінімального кістякового дерева було розроблено програмну реалізацію алгоритму Прима мовою Python. Програма використовує матрицю суміжності зваженого графа варіанта №15 та послідовно визначає ребра з найменшою вагою, які з'єднують уже включені до дерева вершини з невключеними.

Реалізація алгоритму Прима з виведенням проміжних результатів кожного кроку наведена в Лістингу 5.1.

Лістинг 5.1 – Python-код реалізації алгоритму Прима

```
def prim(graph, start=0):
    """
    Алгоритм Прима для побудови мінімального кістякового дерева (МКД).
    Graph - матриця суміжності з вагами ребер
    Start - індекс початкової вершини
    """
    n = len(graph)
    selected = [False] * n
    selected[start] = True

    mst = []
```

```
    total_weight = 0

    print("Початкова вершина:", start + 1)
    print("\n --- Побудова МКД за алгоритмом Прима ---")

    # МКД повинно містити n - 1 ребро
    for step in range(1, n):

        min_weight = float('inf')
        best_u = -1
        best_v = -1

        # Пошук ребра з найменшою вагою
        for u in range(n):
            if selected[u]:
                for v in range(n):

                    # Перевіряємо допустимість ребра
                    if not selected[v] and graph[u][v] > 0:

                        # Знаходимо мінімальну вагу
                        if graph[u][v] < min_weight:
                            min_weight = graph[u][v]
                            best_u = u
                            best_v = v

        # Перевірка зв'язності графа
        if best_u == -1:
            print("Граф незв'язний. Побудова неможлива.")
            return None

        # Додаємо нову вершину до МКД
        selected[best_v] = True

        # Записуємо ребро в порядку номерів вершин
        edge = tuple(sorted((best_u + 1, best_v + 1)))
        mst.append((edge[0], edge[1], min_weight))

        # Обчислюємо загальну вагу
        total_weight += min_weight

        print(f"Крок {step}: ({edge[0]}, {edge[1]}) "
              f"= {min_weight}")
        print(f"Загальна вага: {total_weight}\n")

    return mst, total_weight


# Матриця суміжності графа варіанта №15
G = [
    [0, 8, 2, 4, 3, 0, 0, 0],
    [8, 0, 0, 0, 6, 3, 0, 0],
    [2, 0, 0, 6, 0, 7, 0, 4],
    [4, 0, 6, 0, 1, 0, 0, 0],
    [3, 6, 0, 1, 0, 0, 4, 0],
    [0, 3, 7, 0, 0, 0, 3, 1],
    [0, 0, 0, 0, 4, 3, 0, 5],
    [0, 0, 4, 0, 0, 1, 5, 0]
]

# Моделювання для варіанта №15
result = prim(G, start=0)

if result is not None:
```

```
        mst, total_weight = result

        print("--- Мінімальне кістякове дерево ---")
        for u, v, w in mst:
            print(f"({u}, {v}) = {w}")

    print("\n Загальна вага МКД:", total_weight)
```

У результаті виконання програмного коду, наведеного в Лістингу 5.1, було побудовано мінімальне кістякове дерево для графа варіанта №15. Програма послідовно визначила ребра з мінімальними допустимими вагами та обчислила загальну вагу побудованого дерева. Результати виконання алгоритму Прима наведено в таблиці 5.5.

Таблиця 5.5 – Результати виконання алгоритму Прима

```
Початкова вершина: 1

 --- Побудова МКД за алгоритмом Прима ---
Крок 1: (1, 3) = 2
Загальна вага: 2

Крок 2: (1, 5) = 3
Загальна вага: 5

Крок 3: (4, 5) = 1
Загальна вага: 6

Крок 4: (3, 8) = 4
Загальна вага: 10

Крок 5: (6, 8) = 1
Загальна вага: 11

Крок 6: (2, 6) = 3
Загальна вага: 14

Крок 7: (6, 7) = 3
Загальна вага: 17

--- Мінімальне кістякове дерево ---
(1, 3) = 2
(1, 5) = 3
(4, 5) = 1
(3, 8) = 4
(6, 8) = 1
(2, 6) = 3
(6, 7) = 3

Загальна вага МКД: 17
```

Аналіз результатів:

Порівняння результатів ручної побудови мінімального кістякового дерева (таблиця 5.4) та програмної реалізації алгоритму Прима (таблиця 5.5) показало їх повний збіг. В обох випадках було обрано однакові 7 ребер, які з'єднують усі 8 вершин графа без утворення циклів.

Загальна вага побудованого мінімального кістякового дерева становить 17. Отримані результати підтверджують правильність реалізації алгоритму Прима для графа варіанта №15.

4. Побудова мінімального кістякового дерева за алгоритмом Крускала

Алгоритм Крускала (Kruskal) належить до жадібних алгоритмів і використовується для побудови мінімального кістякового дерева зваженого неорієнтованого зв'язного графа. На відміну від алгоритму Прима, він починає роботу з порожньої множини ребер і поступово формує дерево шляхом вибору ребер із найменшими вагами.

Спочатку всі ребра графа сортуються за неспаданням їхніх ваг. На кожному кроці розглядається чергове ребро, яке додається до дерева лише за умови, що воно не утворює циклу. Для перевірки циклічності використовується структура даних DSU (Disjoint Set Union).

Для реалізації алгоритму використовується граф варіанта №15, представлений на рисунку 5.1. Псевдокод алгоритму Крускала наведено в таблиці 5.6.

Таблиця 5.6 – Псевдокод алгоритму Крускала

```
Kruskal(G)
// Вхідні дані: зважений зв'язний граф G = (V, E)
// Вихідні дані: E_T – множина ребер МКД

// Сортуємо ребра за неспаданням ваги
E_sorted ← SortEdges(E)

// Початкове МКД не містить ребер
E_T ← ∅

// Кожна вершина утворює окрему множину DSU
for each v ∈ V do
    MakeSet(v)

// Переглядаємо ребра за зростанням ваги
for each e = (u, v, w) ∈ E_sorted do
```

```
    // Перевіряємо, чи утворюється цикл
    if Find(u) ≠ Find(v) then

        // Додаємо ребро до МКД
        E_T ← E_T ∪ {e}

        // Об'єднуємо компоненти зв'язності
        Union(u, v)

    // МКД повинно містити |V| - 1 ребро
    if |E_T| = |V| - 1 then
        break

return E_T
```

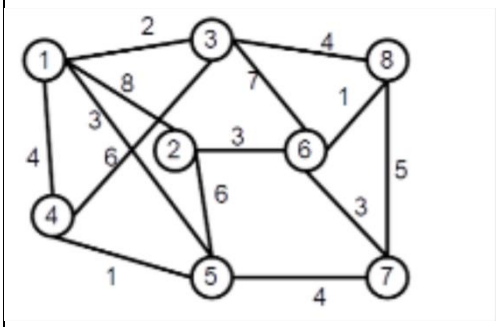
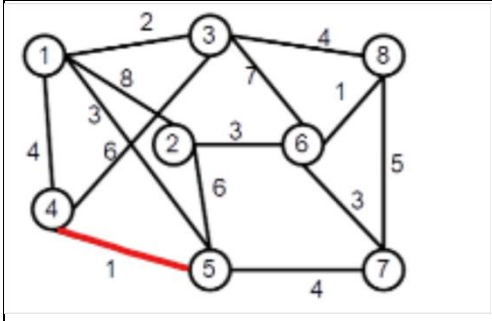
Позначення:

E – множина сортованих неспадному порядку ваг ребер

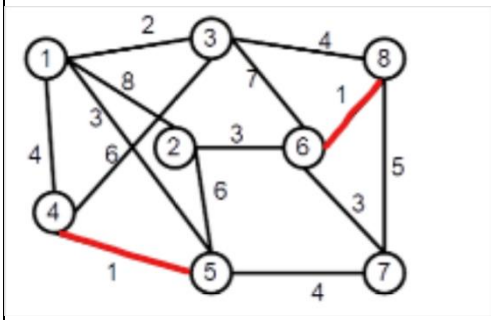
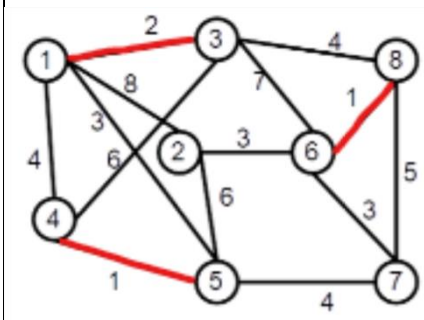
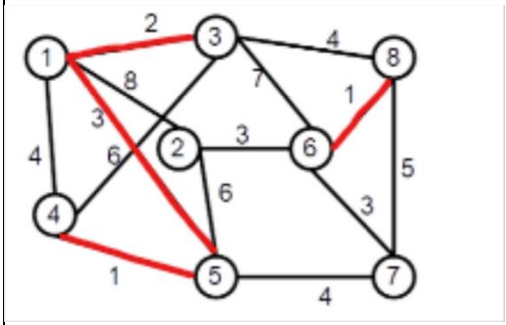
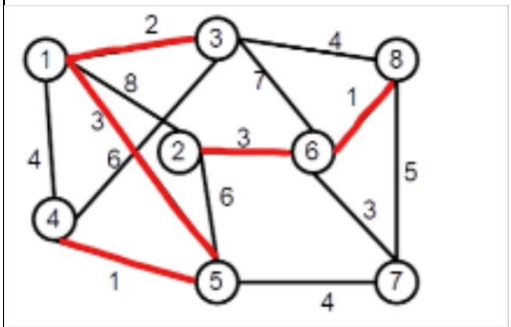
E_T – множина ребер MST

Таблиця 5.7 – Хід побудови мінімального кістякового дерева за алгоритмом Крускала

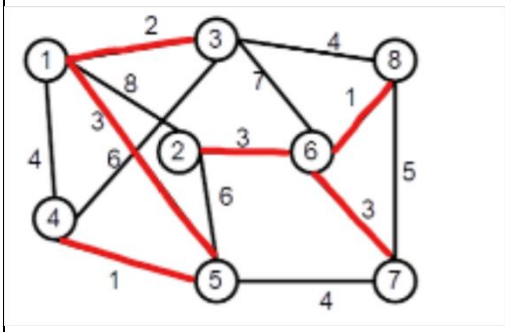
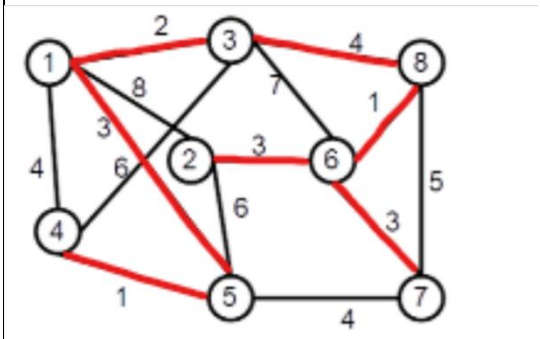
Початок таблиці 5.7

Зображення	E_T	E
	∅	(4,5) = 1 (6,8) = 1 (1,3) = 2 (1,5) = 3 (2,6) = 3 (6,7) = 3 (1,4) = 4 (3,8) = 4 (5,7) = 4 (7,8) = 5 (2,5) = 6 (3,4) = 6 (3,6) = 7 (1,2) = 8
	(4,5) = 1 **Σ = 1**	+ (4,5) = 1 (6,8) = 1 (1,3) = 2 (1,5) = 3 (2,6) = 3 (6,7) = 3 (1,4) = 4 (3,8) = 4 (5,7) = 4 (7,8) = 5 (2,5) = 6 (3,4) = 6 (3,6) = 7 (1,2) = 8

Продовження таблиці 5.7

Зображення	E_T	E
	(4,5) = 1 (6,8) = 1 **Σ = 2**	+ (4,5) = 1 + (6,8) = 1 (1,3) = 2 (1,5) = 3 (2,6) = 3 (6,7) = 3 (1,4) = 4 (3,8) = 4 (5,7) = 4 (7,8) = 5 (2,5) = 6 (3,4) = 6 (3,6) = 7 (1,2) = 8
	(4,5) = 1 (6,8) = 1 (1,3) = 2 **Σ = 4**	+ (4,5) = 1 + (6,8) = 1 + (1,3) = 2 (1,5) = 3 (2,6) = 3 (6,7) = 3 (1,4) = 4 (3,8) = 4 (5,7) = 4 (7,8) = 5 (2,5) = 6 (3,4) = 6 (3,6) = 7 (1,2) = 8
	(4,5) = 1 (6,8) = 1 (1,3) = 2 (1,5) = 3 **Σ = 7**	+ (4,5) = 1 + (6,8) = 1 + (1,3) = 2 + (1,5) = 3 (2,6) = 3 (6,7) = 3 (1,4) = 4 (3,8) = 4 (5,7) = 4 (7,8) = 5 (2,5) = 6 (3,4) = 6 (3,6) = 7 (1,2) = 8
	(4,5) = 1 (6,8) = 1 (1,3) = 2 (1,5) = 3 (2,6) = 3 **Σ = 10**	+ (4,5) = 1 + (6,8) = 1 + (1,3) = 2 + (1,5) = 3 + (2,6) = 3 (6,7) = 3 (1,4) = 4 (3,8) = 4 (5,7) = 4 (7,8) = 5 (2,5) = 6 (3,4) = 6 (3,6) = 7 (1,2) = 8

Кінець таблиці 5.7

Зображення	E_T	E
	(4,5) = 1 (6,8) = 1 (1,3) = 2 (1,5) = 3 (2,6) = 3 (6,7) = 3 **Σ = 13**	+ (4,5) = 1 + (6,8) =1 + (1,3) = 2 + (1,5) = 3 + (2,6) = 3 + (6,7) = 3 (1,4) = 4 (3,8) = 4 (5,7) = 4 (7,8) = 5 (2,5) = 6 (3,4) = 6 (3,6) = 7 (1,2) = 8
	(4,5) = 1 (6,8) = 1 (1,3) = 2 (1,5) = 3 (2,6) = 3 (6,7) = 3 (3,8) = 4 **Σ = 17**	+ (4,5) = 1 + (6,8) = 1 + (1,3) = 2 + (1,5) = 3 + (2,6) = 3 + (6,7) = 3 × (1,4) = 4 + (3,8) = 4 (5,7) = 4 (7,8) = 5 (2,5) = 6 (3,4) = 6 (3,6) = 7 (1,2) = 8

Під час виконання алгоритму Крускала було розглянуто ребро (1,4) з вагою 4. Оскільки вершини 1 і 4 вже з'єднані шляхом 1–5–4, додавання цього ребра призвело б до утворення циклу 1–5–4–1. Тому ребро (1,4) не було включено до мінімального кістякового дерева. Оскільки множина обраних ребер залишилася незмінною, окрему графічну схему цього кроку не наведено.

Мінімальне кістякове дерево: {(4,5) = 1, (6,8) = 1, (1,3) = 2, (1,5) = 3, (6,7) = 3, (2,6) = 3, (3,8) = 4}.

Загальна вага МКД: 1 + 1 + 2 + 3 + 3 + 3 + 4 = 17.

У результаті виконання алгоритму Крускала побудовано мінімальне кістякове дерево, яке містить усі 8 вершин графа та 7 ребер без утворення циклів. Загальна вага отриманого дерева становить 17.

5. Реалізація алгоритму Крускала мовою Python

Для перевірки результатів ручної побудови мінімального кістякового дерева було розроблено програмну реалізацію алгоритму Крускала мовою Python. Програма використовує список ребер зваженого графа варіанта №15, відсортований за неспаданням їхніх ваг.

Для запобігання утворенню циклів застосовується структура даних DSU (Disjoint Set Union), яка дозволяє перевіряти належність вершин до однієї компоненти зв'язності. Програма послідовно визначає допустимі ребра, формує мінімальне кістякове дерево та обчислює його загальну вагу.

Python-код реалізації алгоритму Крускала з виведенням результатів кожного кроку наведено в Лістингу 5.2.

Лістинг 5.2 – Python-код реалізації алгоритму Крускала

```
def find(parent, v):
    """
    Пошук представника множини для вершини v.
    Використовується стиснення шляху.
    """
    if parent[v] != v:
        parent[v] = find(parent, parent[v])

    return parent[v]


def union(parent, rank, u, v):
    """
    Об'єднання двох компонент зв'язності.
    Повертає False, якщо утворюється цикл.
    """
    root_u = find(parent, u)
    root_v = find(parent, v)

    # Перевірка на утворення циклу
    if root_u == root_v:
        return False

    # Об'єднання множин за рангом
    if rank[root_u] < rank[root_v]:
        parent[root_u] = root_v

    elif rank[root_u] > rank[root_v]:
        parent[root_v] = root_u

    else:
        parent[root_v] = root_u
```

```
            rank[root_u] += 1

        return True


    def kruskal(n, edges):
        """
        Алгоритм Крускала для побудови МКД.
        n: кількість вершин графа
        edges: список ребер (u, v, вага)
        """
        # Сортування ребер за вагою та номерами вершин
        edges = sorted(edges, key=lambda e: (e[2], e[0], e[1]))

        # Ініціалізація структури DSU
        parent = list(range(n + 1))
        rank = [0] * (n + 1)

        mst = []
        total_weight = 0

        print("--- Побудова МКД за алгоритмом Крускала ---")

        # Переглядаємо ребра за зростанням ваги
        for step, (u, v, w) in enumerate(edges, start=1):

            # Додаємо ребро, якщо воно не утворює цикл
            if union(parent, rank, u, v):
                mst.append((u, v, w))
                total_weight += w

                print(f"Крок {step}: ({u}, {v}) = {w} "
                      f"— ребро додано")
                print(f"Загальна вага: {total_weight}\n")

            else:
                print(f"Крок {step}: ({u}, {v}) = {w} "
                      f"— пропущено (утворюється цикл)\n")

            # МКД повинно містити n - 1 ребро
            if len(mst) == n - 1:
                break

        # Перевірка зв'язності графа
        if len(mst) != n - 1:
            print("Граф незв'язний. МКД не побудовано.")
            return None

        return mst, total_weight


    # Ребра графа варіанта №15: (вершина, вершина, вага)
    edges = [
        (1, 2, 8),
        (1, 3, 2),
        (1, 4, 4),
        (1, 5, 3),
        (2, 5, 6),
        (2, 6, 3),
        (3, 4, 6),
        (3, 6, 7),
        (3, 8, 4),
        (4, 5, 1),
        (5, 7, 4),
```

```
        (6, 7, 3),
        (6, 8, 1),
        (7, 8, 5)
    ]

    # Моделювання для варіанта №15
    result = kruskal(8, edges)

    if result is not None:
        mst, total_weight = result

        print("--- Мінімальне кістякове дерево ---")

        for u, v, w in mst:
            print(f"({u}, {v}) = {w}")

print("\n Загальна вага МКД:", total_weight)
```

У результаті виконання програмного коду, наведеного в Лістингу 5.2, було побудовано мінімальне кістякове дерево графа варіанта №15 за алгоритмом Крускала. Програма послідовно розглядала ребра за зростанням їхніх ваг, перевіряла можливість утворення циклів та обчислювала загальну вагу побудованого дерева. Результати виконання алгоритму наведено в таблиці 5.8.

Таблиця 5.8 – Результати виконання алгоритму Крускала

```
--- Побудова МКД за алгоритмом Крускала ---
Крок 1: (4, 5) = 1 — ребро додано
Загальна вага: 1

Крок 2: (6, 8) = 1 — ребро додано
Загальна вага: 2

Крок 3: (1, 3) = 2 — ребро додано
Загальна вага: 4

Крок 4: (1, 5) = 3 — ребро додано
Загальна вага: 7

Крок 5: (2, 6) = 3 — ребро додано
Загальна вага: 10

Крок 6: (6, 7) = 3 — ребро додано
Загальна вага: 13

Крок 7: (1, 4) = 4 — пропущено (утворюється цикл)

Крок 8: (3, 8) = 4 — ребро додано
Загальна вага: 17

--- Мінімальне кістякове дерево ---
(4, 5) = 1
(6, 8) = 1
```

```
(1, 3) = 2
(1, 5) = 3
(2, 6) = 3
(6, 7) = 3
(3, 8) = 4

Загальна вага МКД: 17
```

Аналіз результатів:

Порівняння результатів ручної побудови мінімального кістякового дерева (таблиця 5.7) та програмної реалізації алгоритму Крускала (таблиця 5.8) показало їхній збіг. В обох випадках було отримано однакову множину з 7 ребер, які з'єднують усі 8 вершин графа без утворення циклів.

Під час виконання програми ребро (1,4) з вагою 4 було відхилено, оскільки його додавання призвело б до утворення циклу. Послідовність додавання ребер під час ручної побудови та програмного виконання алгоритму Крускала повністю збігається.

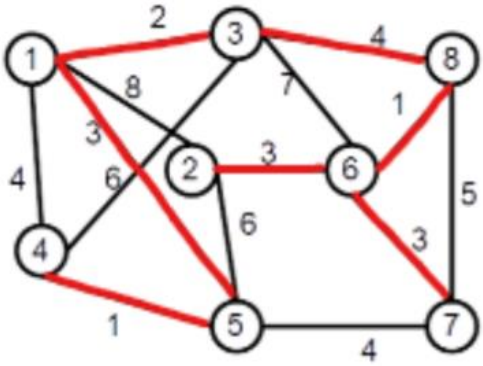
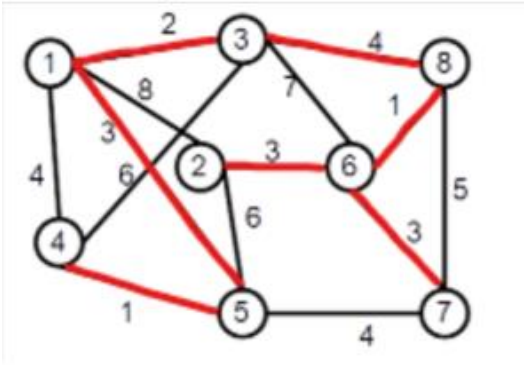
Загальна вага побудованого мінімального кістякового дерева становить 17, що підтверджує правильність програмної реалізації алгоритму Крускала для графа варіанта №15.

6. Порівняльний аналіз алгоритмів Прима та Крускала

Для порівняння результатів роботи алгоритмів Прима та Крускала було проаналізовано мінімальні кістякові дерева, отримані під час ручної побудови та програмного виконання. Обидва алгоритми використовували однаковий зважений граф варіанта №15, що містить 8 вершин і 14 ребер.

У результаті виконання алгоритмів було побудовано мінімальні кістякові дерева, кожне з яких містить 7 ребер. Порівняння отриманих результатів наведено в таблиці 5.9.

Таблиця 5.9 – Порівняння результатів алгоритмів Прима та Крускала

Алгоритм Прима	*Алгоритм Крускала*
	
(1,3) = 2 (1,5) = 3 (4,5) = 1 (3,8) = 4 (6,8) = 1 (2,6) = 3 (6,7) = 3	(4,5) = 1 (6,8) = 1 (1,3) = 2 (1,5) = 3 (2,6) = 3 (6,7) = 3 (3,8) = 4
Загальна вага: 17	Загальна вага: 17

За результатами порівняння (таблиця 5.9) встановлено, що алгоритми Прима та Крускала побудували однакове мінімальне кістякове дерево із загальною вагою 17. В обох випадках до дерева включено ті самі 7 ребер, які з'єднують усі 8 вершин графа без утворення циклів.

Основна відмінність полягає у способі побудови дерева. Алгоритм Прима починає роботу з початкової вершини та поступово розширює дерево, додаючи ребро з найменшою допустимою вагою. Алгоритм Крускала розглядає всі ребра за неспаданням їхніх ваг і додає їх за умови відсутності циклів.

Під час роботи алгоритму Крускала ребро (1,4) з вагою 4 було відхилено, оскільки його додавання призвело б до утворення циклу. Незважаючи на різну послідовність вибору ребер, кінцеві результати обох алгоритмів повністю збігаються.

Отже, проведене дослідження підтвердило правильність ручної та програмної побудови мінімального кістякового дерева для графа варіанта №15 за обома алгоритмами.

Висновок:

Під час виконання лабораторної роботи було досліджено, реалізовано та протестовано алгоритми Прима (Prim) і Крускала (Kruskal) для побудови мінімального кістякового дерева зваженого неорієнтованого графа. Для графа варіанта №15, що містить 8 вершин і 14 ребер, було побудовано матрицю суміжності з вагами та списки суміжності.

Шляхом ручного трасування та графічної візуалізації досліджено послідовність побудови мінімального кістякового дерева за обома алгоритмами. Також було розроблено та протестовано їхні програмні реалізації мовою Python. Результати програмного виконання повністю збіглися з результатами ручної побудови: в обох випадках отримано мінімальне кістякове дерево із 7 ребер та загальною вагою 17.

Порівняльний аналіз показав, що алгоритм Прима поступово розширює дерево від початкової вершини, тоді як алгоритм Крускала послідовно розглядає ребра за зростанням їхніх ваг, уникаючи утворення циклів. Незважаючи на відмінності у принципах роботи та послідовності вибору ребер, обидва алгоритми сформували однакове мінімальне кістякове дерево.

Таким чином, було набуто практичних навичок представлення графів, побудови мінімальних кістякових дерев, трасування алгоритмів та перевірки правильності отриманих результатів за допомогою програмного моделювання.

Посилання на GitHub:

Посилання на онлайн компілятор: https://coddy.tech/playground/ru/python